Practical 5: Python Libraries and Basic Web Scraping

Aim

To study and implement basic web scraping using Python libraries requests and BeautifulSoup.

Theory

Web scraping is the process of programmatically extracting information from websites by fetching their raw HTML and parsing out the required data, commonly used for data collection, price monitoring, and research when no structured API is available; it should always be done in compliance with a site's terms of service and robots.txt. The requests library is a popular third-party HTTP client used to send GET/POST requests and retrieve a page's HTML content along with metadata like status codes and headers, and is generally preferred over the lower-level urllib for its simpler, more readable API. BeautifulSoup, from the bs4 package, is an HTML/XML parsing library that builds a navigable parse tree from raw markup and provides convenient methods to search and traverse it, such as tag access via dot notation (soup.title, soup.h1), find() and find_all() to locate elements by tag name, class, or attribute, and .text/.string to extract the readable text content of an element, abstracting away the need to manually write regular expressions against HTML.

Algorithm

1. Import the requests library and BeautifulSoup from bs4.
2. Send a GET request to the target URL to fetch the HTML page.
3. Parse the HTML content using BeautifulSoup with the 'html.parser'.
4. Extract and print the page title using soup.title.string.
5. Extract and print the first heading using soup.h1.text.
6. Find all paragraph tags using soup.find_all('p') and display their count and content.

Program

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url, timeout=5)
soup = BeautifulSoup(response.text, "html.parser")

print("Page Title:", soup.title.string)
print("First Heading:", soup.h1.text)
paragraphs = soup.find_all("p")
print("Number of paragraphs:", len(paragraphs))
print("First paragraph:", paragraphs[0].text)
```

Output

```
Page Title: Example Domain
First Heading: Example Domain
Number of paragraphs: 2
First paragraph: This domain is for use in documentation examples without needing permission. Avoid use in..
```

Result / Conclusion

Basic web scraping was successfully implemented, extracting the title, heading, and paragraph content from a web page.

Viva Questions

Q1. What is web scraping and what are its legal/ethical considerations?

Ans: Web scraping is the automated extraction of data from websites by fetching HTML and parsing it programmatically. Ethical/legal considerations: always check the site's robots.txt file (which lists paths scrapers should not access) and Terms of Service; avoid overloading servers with rapid requests; do not scrape personal or copyrighted data without permission; use official APIs when available. Violating these can lead to IP bans, legal action, or violation of laws like the CFAA.

Q2. What is the difference between soup.find() and soup.find_all()?

Ans: soup.find(tag) returns the first matching element as a Tag object (or None if not found). soup.find_all(tag) returns a list of ALL matching elements (empty list if none found). find() is equivalent to find_all(..., limit=1)[0]. Both accept tag name, CSS class, id, or any attribute as search criteria. find() is used when you expect a unique element; find_all() when you need all occurrences.

Q3. What parsers can BeautifulSoup use, and which is recommended?

Ans: 'html.parser' is Python's built-in HTML parser — no extra install needed, reasonably fast, lenient with malformed HTML. 'lxml' is a fast C-based parser (pip install lxml) — best performance for large pages. 'html5lib' (pip install html5lib) parses exactly like a browser — slowest but most accurate for malformed HTML. For most tasks, 'html.parser' is sufficient; use 'lxml' for performance-critical scraping.

Q4. What is the difference between .text and .string in BeautifulSoup?

Ans: .text (or .get_text()) returns all the text content of a tag and all its descendants concatenated as a single string, stripping HTML tags. .string returns the NavigableString of a tag ONLY if it contains a single string with no nested tags; it returns None if the tag has multiple children. Use .text for general extraction; .string only when you are certain the tag has a single text node.

Q5. How can you handle a failed HTTP request in web scraping?

Ans: Check response.status_code: 200 = OK, 404 = Not Found, 403 = Forbidden, 429 = Too Many Requests. Use response.raise_for_status() to raise an HTTPError for 4xx/5xx responses. Wrap requests.get() in a try-except block for connection errors (requests.exceptions.ConnectionError, Timeout). Add retry logic with exponential backoff for rate-limited sites. Always set a timeout parameter to avoid hanging indefinitely.